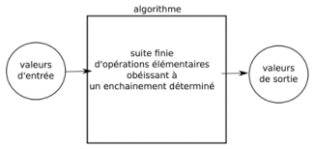
NSI S4 : Algorithmique



Un algorithme est une série d'actes ou d'opérations élémentaires qu'il faut exécuter en séquence pour accomplir une tâche quelconque, en suivant un enchaînement strict.
Lorsqu'il sera demandé d'élaborer un algorithme, la méthode pour atteindre cet objectif sera de rédiger en français la succession des opérations élémentaires (phases courtes et précises) puis de passer à une écriture conventionnelle appelée pseudo-code ou algorigramme.

1. Organisation d'un algorithme :

- L'en-tête : Nom de l'algorithme. description du traitement effectué, données en entrée (paramètres, arguments), données en sortie (résultats).
- La partie déclarative : Liste des "objets" utilisés par l'algorithme (constantes, variables).
- Le corps : Une suite d'instructions délimitée par les mots "début" en "fin".

Le pseudo-code consiste à exprimer en langage naturel, mais selon une disposition particulière et des mots choisis, les différentes opérations constituant l'algorithme, conformément au code donné dans le tableau qui suit.

```
fonction Surface

Calcule la surface d'un disque
à partir de son rayon

Paramètre :
      rayon : réel
Résultat :
      surface : réel

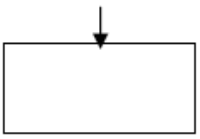
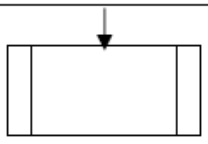
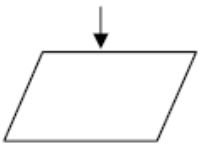
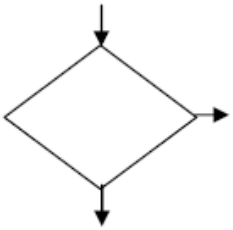
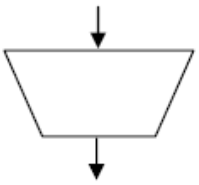
Pi = 3,14
surface : réel

début
      surface <- Pi * rayon * rayon
      retourner surface
fin
```

Remarque :
Le pseudo-code se prête mieux que l'algorigramme à la description des structures complexes existant dans les langages de haut niveau.

Mots et symboles du pseudo-code	Opérations réalisées
Début	Début de l'algorithme, permet de le nommer
Fin	Fin de l'algorithme
Faire	Exécution d'une opération
Entrer	Acquisition ou chargement d'une donnée
Sortir	Edition ou sauvegarde d'un résultat
Retourner	Retourner le résultat d'une fonction
←	Affectation d'une valeur à une variable
Symboles d'opérateur (+-*/ ET OU,...)	Opérations arithmétiques ou logiques
Aller à	Branchement inconditionnel (déconseillé)
Si…alors…[sinon]	Branchement inconditionnel
Selon cas…[autrement]	Branchement conditionnel généralisé
Tant que…faire…	} Répétition conditionnelle
Répéter…jusqu'à…	
Pour…=…à…	Répétition contrôlée

L'algorigramme est la représentation graphique de l'algorithme, il permet de représenter chaque opération élémentaire au moyen d'un symbole graphique normalisé.

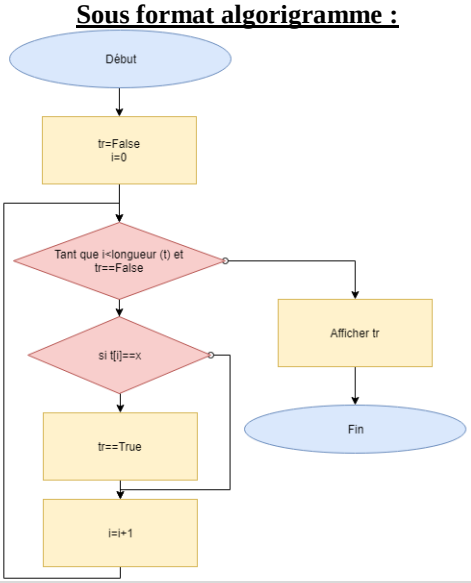
 début / fin	 Traitement	 Sous programme	 Entrée / sortie
 Test / condition	 Opération manuelle	 Symbole de renvoi utilisé 2 fois pour assurer la continuité lorsqu'une partie de la ligne de liaison n'est pas représentée.	 Commentaires : symbole utilisé pour donner des indications marginales

2. Premier algorithme : est-ce que x est dans un tableau t ?

Sous format texte :

```
DEBUT
tr ← False
i ← 0
tant que i<longueur(t) et que tr==False:
  si t[i]==x:
    tr ← True
  fin si
  i ← i+1
fin tant que
Afficher la valeur de tr
FIN
```

Sous format algorigramme :



I) Tester à la main l'algorithme avec t = [12 ,0 ,5 ,10] et x=5

II) Créer le programme python où t sera une constante déclarée en début de programme et x demandée à l'utilisateur (input()), sauvegarder sous Exo2.py puis tester avec x=5 et x=1.

Complexité d'un algorithme

La notion de complexité d'un algorithme représente l'efficacité de cet algorithme.
Nous allons étudier uniquement la complexité en temps qui est liée au nombre d'opérations élémentaires qui doivent être exécutées afin de résoudre un problème donné. L'évaluation de ce nombre d'opérations élémentaires n'est pas chose facile, on rencontre souvent des cas litigieux.
Exemple avec : "est-ce que x est dans le tableau t ?".
Il y a 2 cas à traiter :

- L'entier recherché est bien présent dans le tableau, il se trouve à la position d'index i
- L'entier recherché n'est pas présent dans le tableau

Pour calculer la complexité, on se place toujours dans le cas le plus défavorable, ici le deuxième.
On va rechercher dans l'algorithme, le test qui sera exécuté le plus de fois :

```
DEBUT
1 fois   tr ← False
1 fois   i ← 0
n+1 fois tant que i<=longueur(t) et que tr==False:
n fois     si t[i]==x:
1 fois       tr ← True
           fin si
n fois     i ← i+1
         fin tant que
1 fois   afficher la valeur de tr
FIN
```

On remarque que c'est n+1 fois

On dira que la complexité de cet algorithme est égale à O(n).
Si nous avions eu n^2+3n+2 nous aurions $O(n^2)$……

3. Le plus grand entier du tableau t ?

III) Ecrire et tester à la main l'algorithme qui donne le plus grand entier de t avec t = [12,5,10,22].

IV) Déterminer la complexité de cet algorithme.

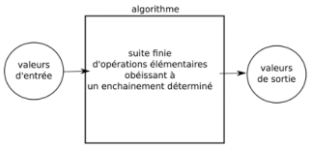
V) Créer le programme python où t sera une constante déclarée en début de programme, sauvegarder sous Exo5.py puis tester.

4. La moyenne du tableau t ?

VI) Ecrire et tester à la main l'algorithme qui donne la moyenne de t avec t = [12,5,10,22].

VII) Déterminer la complexité de cet algorithme.

VIII) Créer le programme python où t sera une constante déclarée en début de programme, sauvegarder sous Exo8.py puis tester.



5. Fonction du tableau t :

Nous avons vue trois exercices sur les tableaux dans trois programmes différents. Ces exercices pourraient être assimilés à des fonctions que l'on pourrait réutiliser dans un programme. Pour cela il suffit de les coder en tant que tel.
Pour créer une fonction sous python, il suffit de mettre l'instruction def suivi du nom de la fonction et des paramètres entre parenthèses :

```
def NomDeLaFonction(parametre1, parametre2, parametre3,...):
    """ Documentation
    qu'on peut écrire
    sur plusieurs lignes """# docstring entouré de 3 guillemets (ou apostrophes)

    "bloc d'instructions"# attention à l'indentation

    return resultat      #la fonction retourne le contenu de la variable resultat
```

Exemple pour la fonction x dans tableau(t)

```
tableau=[12,0,5,10]

def x_dans_t (tab):
    """On recherche si x est dans le tableau tab"""
    tr=False
    i=0
    x=int(input("Donner le nombre à rechercher :"))
    while(i<len(tab) and tr==False):
        if(tab[i]==x):
            tr=True
        i=i+1
    return(tr)

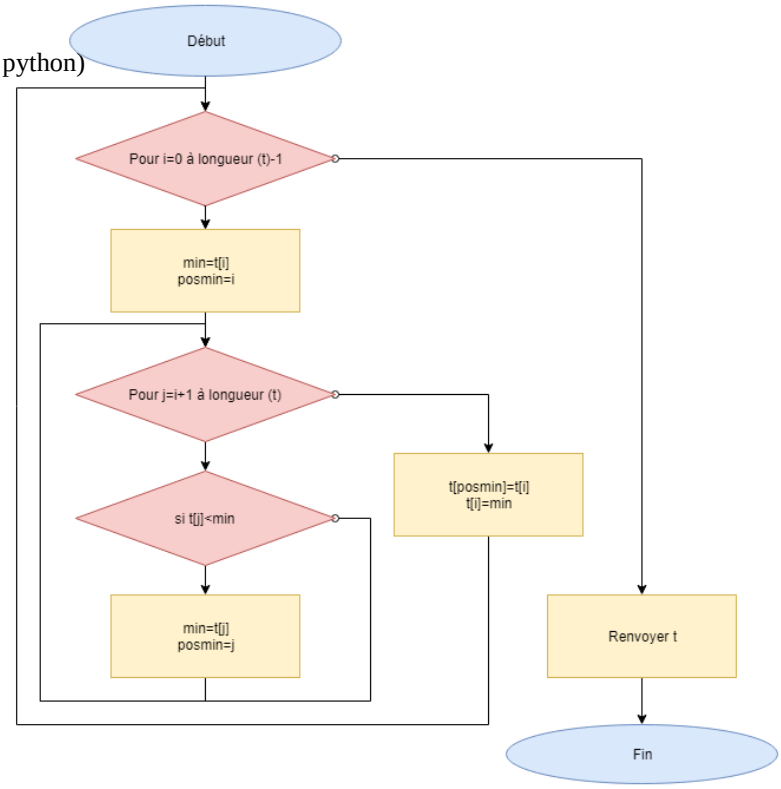
print("Est-ce que x est dans le tableau ?  ", x_dans_t(tableau))
```

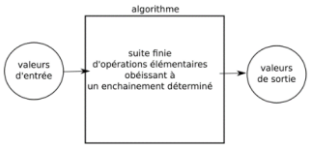
IX) Ecrire et tester ce programme puis rajouter les deux autres fonctions « max_de_t » et « moyenne_de_t » puis sauver sous Exo9.py. Appeler les fonctions et afficher les résultats pour le tableau[12,5,10,22]

6. Tri du tableau par sélection du minimum :

L'algorithme de cette fonction est le suivant :

```
DEBUT
  Pour i variant de 0 à n=longueur(t)-1 (exclu sous python)
      min ← t[i]
      posmin ← i
      Pour j variant de i+1 à n (exclu sous python)
                Si t[j]<min
                          min ← t[j]
                          posmin ← j
                Fin Si
      Fin Pour
      #Echanger t[i] et min
      t[posmin] ←t[i]
      t[i] ← min
  Fin Pour
  Renvoyer t
FIN
```





X) Tester cet algorithme avec le tableau [5,9,-1,2,7] en remplissant le tableau ci-dessous pour chaque valeur de i (vous préciserez avec une double flèches les deux éléments échangés).

i	t				
0	5	9	-1	2	7
1	-1	9	5	2	7
2					
3					

Pour déterminer la complexité de cet algorithme, on se rend compte que l'opération qui va être faite le plus de fois est SI t[j]<min.

i	j	Nb total pour chaque i
0	1 à n-1	n-1
1	2 à n-1	n-2
2	3 à n-1	n-3
.. ..	.. ..	……. …….
n-3	n-2 à n-1	2
n-2	n-1 à n-1	1
Soit un total de 1+2+…..+(n-3)+(n-2)+(n-1)=n*(n-1)/2		

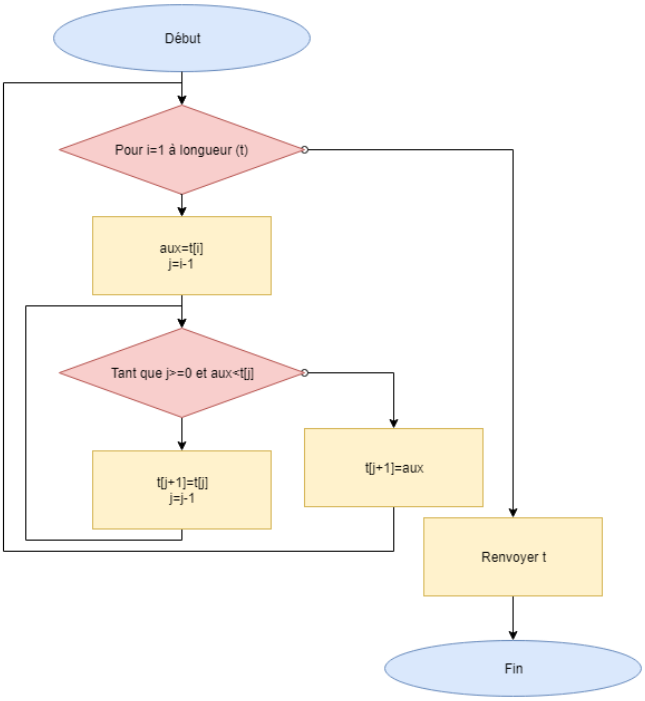
Pour n très grand $n*(n-1)/2 \cong n^2/2$ donc on dira que la complexité est en $O(n^2)$.

XI) Coder cet algorithme sous forme de fonction Tri_min et tester la avec le tableau [5,9,-1,2,7] . Sauvegarder sous Exo11.py

7. Tri du tableau par insertion :

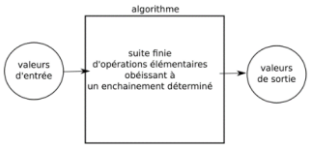
L'algorithme de cette fonction est le suivant :

DEBUT
  Pour i variant de 1 à n=longueur(t) (exclu sous python)
    aux ← t[i]
    j ← i-1
    Tant que j≥0 et aux<t[j]
        t[j+1] ← t[j]
        j ← j-1
    fin tant que
    t[j+1] ←aux
  Fin Pour
  Renvoyer t
FIN



XII) Tester cet algorithme avec le tableau [6,4,5,1,9] en remplissant le tableau ci-dessous pour chaque valeur de i et j(vous préciserez avec une double flèches les deux éléments échangés).

i	aux	j=3	j=2	j=1	j=0	Tableau avant prochaine itération de i
1	4				66519	46519
2	5			46619		45619



Pour déterminer la complexité de cet algorithme, on se rend compte que l'opération qui va être faite le plus de fois est j≥0.

i	j	Nb total pour chaque i
1	0 à -1	2
2	1 à -1	3
.. ..	.. ..	……. …….
n-2	n-3 à -1	n-1
n-1	n-2 à -1	n
Soit un total de 2+3…..+(n-1)+n=(n+2)*(n-1)/2		

Pour n très grand $(n+2)*(n-1)/2 \cong n^2/2$ donc on dira que la complexité est en $O(n^2)$.

XIII) Coder cet algorithme sous forme de fonction Tri_Insertion et tester la avec le tableau [6,4,5,1,9] . Sauvegarder sous Exo13.py

8. Les invariants de boucle

A priori, les algorithmes de tri par insertion et de tri par sélection "fonctionnent" correctement, on dit que ces algorithmes sont corrects.
Il se pourrait très bien que dans une situation donnée notre algorithme ne donne pas le résultat attendu.
Il existe un moyen de démonter la correction d'un algorithme. Si nous arrivons à démontrer la correction de l'algorithme de tri, nous pourrons alors affirmer que, quel que soit le tableau donné en entrée, nous obtiendrons bien en sortie ce même tableau, mais trié.
Pour démontrer la correction de l'algorithme de tri par insertion, il faut utiliser un "invariant de boucle"
On appelle **invariant de boucle, une propriété qui est vraie avant et après chaque itération**.
La démonstration doit se faire en 3 étapes :

- INITIALISATION : on doit montrer que l'invariant de boucle est vrai avant la première itération de la boucle
- CONSERVATION : on doit montrer que si l'invariant de boucle est vrai avant une itération de la boucle, il le reste avant l'itération suivante.
- TERMINAISON : une fois la boucle terminée, l'invariant fournit une propriété utile qui aide à montrer la correction de l'algorithme.

Prenons un exemple, le tri par insertion :

```
DEBUT
  Pour i variant de 1 à n=longueur(t)
      aux ← t[i]
      j ← i-1
      Tant que j≥0 et aux<t[j]
                t[j+1] ← t[j]
                j ← j-1
      fin tant que
      t[j+1] ←aux
  Fin Pour
FIN
```

i	aux	Tableau avant prochaine itération de i
Initialisation i=0		**6** 4 5 1 9
1	4	**4 6** 5 1 9
2	5	**4 5 6** 1 9
3	1	**1 4 5 6** 9
4	6	**1 4 5 6 9**

A chaque itération de i (de la boucle pour), nous devons montrer que le tableau t[0….i] est trié : c'est l'invariant de boucle.

- INITIALISATION : on a i=1 donc t[0..1] ne comporte qu'un élément, il est forcément trié.
- CONSERVATION : si on regarde l'algorithme, on déplace tous les éléments vers la droite tant que l'élément i est plus petit que l'élément j (jusqu'à ce que l'on trouve la bonne place dans la partie triée) donc forcément le nouveau tableau t[0…i] est trié puisque t[0 .. 1] est trié, t[0…2] le sera et ainsi de suite jusqu'à t[0….i]. Donc l'invariant le tableau t[0….i] est trié avant est vrai et le reste après chaque itération.
- TERMINAISON : la boucle se termine, i = n donc notre tableau est t[0….n] qui correspond au tableau d'origine mais trié.

Cette démonstration nous permet d'affirmer que l'algorithme de tri par insertion est correct.

XIV) Démontrer que l'invariant de boucle pour le tri par sélection du minimum est : le tableau t[0..j] est trié où j=i+1.

i	t				
0	5	9	-1	2	7
1	-1	9	5	2	7
2	-1	2	5	9	7
3	-1	2	5	9	7
	-1	2	5	7	9